

Path with the Minimum Sum of Two Largest Edges

Topic.

Graph algorithms, path optimization, modified Dijkstra algorithm.

Statement.

Given an undirected weighted graph with positive edge weights. Find a path from s to t such that the sum of the two largest edge weights on the path is minimized. The required running time is almost linear.

Solution.

Let gr be the adjacency list of the weighted graph.

$gr[v]$ is the list of pairs (u, w) where there is an edge $v - u$ of weight w .

All edge weights are positive.

Create an array $best$ of size n .

$$best[v] = (a, b),$$

where $a \geq b$ are the two largest edge weights on the best path from s to v .

Initially, for all v :

$$best[v] = (+\infty, +\infty).$$

$$best[s] = (0, 0).$$

Create a min-heap pq .

In pq we store vertices v with key

$$best[v].first + best[v].second.$$

Push s into pq .

Define the function $\text{merge}(a, b, w)$:

$$\text{merge}(a, b, w) = \begin{cases} (w, a), & \text{if } w \geq a, \\ (a, w), & \text{if } w \geq b, \\ (a, b), & \text{otherwise.} \end{cases}$$

While pq is not empty:

take $v = pq.pop()$.

For each pair (u, w) from $gr[v]$:

$$cand = \text{merge}(best[v], w).$$

If

$$cand.first + cand.second < best[u].first + best[u].second,$$

then

$$best[u] = cand$$

and push u into pq .

The answer is

$$best[t].first + best[t].second.$$

Complexity.

Each relaxation is done in $O(1)$.

Operations with `priority_queue` take

$$O((n + m) \log n).$$

Therefore, the total running time is

$$O((n + m) \cdot \text{poly}(\log n)).$$